

AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY



Department of Computer Science and Engineering

Course No: CSE 2104

Course Title: Data Structures Lab

Assignment No: 04

Date of Submission: 26/07/2026

Submitted by,

Name: Sheikh Zubayer Kabir

Student ID: 00724205101021

Lab Section: A1

Academic Semester: Fall 2025

Program: BSc in CSE

Question : Implementing the functions of singly linked list in a single .cpp file.

Solution:

Linked List : A linked list is a fundamental data structure in computer science. It mainly allows efficient insertion and deletion operations compared to [arrays](#). Like arrays, it is also used to implement other data structures like stack, queue and deque.

A linked list is a type of linear data structure individual items are not necessarily at contiguous locations. The individual items are called nodes and connected with each other using links.

Code:

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node
```

```
{
```

```
    int data;
```

```
    Node *next;
```

```
};
```

```
Node *createNode(int value);
```

```
void insertFirst(Node *&head, int value);
```

```
void insertLast(Node *&head, int value);
```

```
void insertByPosition(Node *&head, int value, int position);
```

```
void insertByValue(Node *&head, int value, int afterValue);
```

```
void deleteFirst(Node *&head);
```

```
void deleteLast(Node *&head);
```

void deleteByPosition(Node *&head, int position);

void deleteByValue(Node *&head, int value);

void printing(Node *head);

void searching(Node *head, int value);

void last_node(Node *head);

void previous_of_last_node(Node *head);

void list_size(Node *head);

void reversePrint(Node *head);

void reversePrintHelper(Node *node, Node *head);

int main()

{

Node *head = nullptr;

cout << "----- Insert Operations -----\\n";

insertLast(head, 10);

insertLast(head, 20);

insertLast(head, 30);

insertFirst(head, 5);

printing(head);

insertByPosition(head, 15, 2);

printing(head);

insertByValue(head, 25, 20);

printing(head);

```
cout << "\n----- Search / Info Operations ----- \n";
```

```
searching(head, 25);
```

```
searching(head, 100);
```

```
last_node(head);
```

```
previous_of_last_node(head);
```

```
list_size(head);
```

```
reversePrint(head);
```

```
cout << "\n----- Delete Operations ----- \n";
```

```
deleteFirst(head);
```

```
printing(head);
```

```
deleteLast(head);
```

```
printing(head);
```

```
deleteByPosition(head, 1);
```

```
printing(head);
```

```
deleteByValue(head, 25);
```

```
printing(head);
```

```
list_size(head);
```

```
cout << "\n----- Final State ----- \n";
```

```
printing(head);
```

```
reversePrint(head);
```

```
while (head != nullptr)
```

```
{  
    Node *temp = head;  
    head = head->next;  
    delete temp;  
}
```

```
    return 0;  
}
```

```
Node *createNode(int value)  
{  
    Node *newNode = new Node();  
    newNode->data = value;  
    newNode->next = nullptr;  
    return newNode;  
}
```

```
void insertFirst(Node *&head, int value)  
{  
    Node *newNode = createNode(value);  
    newNode->next = head;  
    head = newNode;  
    cout << value << " inserted at first.\n";  
}
```

```
void insertLast(Node *&head, int value)  
{  
    Node *newNode = createNode(value);
```

```

if (head == nullptr)
{
    head = newNode;

    cout << value << " inserted at last.\n";

    return;
}

Node *temp = head;
while (temp->next != nullptr)
{
    temp = temp->next;
}

temp->next = newNode;

cout << value << " inserted at last.\n";
}

void insertByPosition(Node *&head, int value, int position)
{
    if (position < 0)
    {
        cout << "Invalid position.\n";

        return;
    }

    if (position == 0)
    {
        insertFirst(head, value);

        return;
    }

```

```

Node *temp = head;
int index = 0;
while (temp != nullptr && index < position - 1)
{
    temp = temp->next;
    index = index + 1;
}

if (temp == nullptr)
{
    cout << "Position out of range.\n";
    return;
}

Node *newNode = createNode(value);
newNode->next = temp->next;
temp->next = newNode;
cout << value << " inserted at position " << position << ".\n";
}

void insertByValue(Node *&head, int value, int afterValue)
{
    Node *temp = head;
    while (temp != nullptr && temp->data != afterValue)
    {
        temp = temp->next;
    }
}

```

```
if (temp == nullptr)
{
    cout << afterValue << " not found in the list. Insertion failed.\n";
    return;
}
```

```
Node *newNode = createNode(value);
newNode->next = temp->next;
temp->next = newNode;
cout << value << " inserted after " << afterValue << ".\n";
}
```

```
void deleteFirst(Node *&head)
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete.\n";
        return;
    }
    Node *temp = head;
    head = head->next;
    cout << temp->data << " deleted from first.\n";
    delete temp;
}
```

```
void deleteLast(Node *&head)
{
    if (head == nullptr)
```



```

{
    cout << "List is empty. Nothing to delete.\n";
    return;
}
if (head->next == nullptr)
{
    cout << head->data << " deleted from last.\n";
    delete head;
    head = nullptr;
    return;
}
Node *temp = head;
while (temp->next->next != nullptr)
{
    temp = temp->next;
}
cout << temp->next->data << " deleted from last.\n";
delete temp->next;
temp->next = nullptr;
}

```

```

void deleteByPosition(Node *&head, int position)
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete.\n";
        return;
    }
}

```

```
if (position < 0)
{
    cout << "Invalid position.\n";
    return;
}
```

```
if (position == 0)
{
    deleteFirst(head);
    return;
}
```

```
Node *temp = head;
int index = 0;
while (temp != nullptr && index < position - 1)
{
    temp = temp->next;
    index = index + 1;
}
```

```
if (temp == nullptr || temp->next == nullptr)
{
    cout << "Position out of range.\n";
    return;
}
```

```
Node *nodeToDelete = temp->next;
temp->next = nodeToDelete->next;
cout << nodeToDelete->data << " deleted from position " << position << ".\n";
```

```
    delete nodeToDelete;
}
```

```
// (6b) Delete by value
```

```
void deleteByValue(Node *&head, int value)
```

```
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete.\n";
        return;
    }
    if (head->data == value)
    {
        deleteFirst(head);
        return;
    }
```

```
    Node *temp = head;
```

```
    while (temp->next != nullptr && temp->next->data != value)
```

```
    {
        temp = temp->next;
    }
```

```
    if (temp->next == nullptr)
```

```
    {
        cout << value << " not found in the list.\n";
        return;
    }
```

```
    Node *nodeToDelete = temp->next;
    temp->next = nodeToDelete->next;
    cout << value << " deleted from the list.\n";
    delete nodeToDelete;
}
```

```
void printing(Node *head)
{
    if (head == nullptr)
    {
        cout << "List is empty.\n";
        return;
    }
    Node *temp = head;
    cout << "List: ";
    while (temp != nullptr)
    {
        cout << temp->data;
        if (temp->next != nullptr)
        {
            cout << " -> ";
        }
        temp = temp->next;
    }
    cout << "\n";
}
```

```

void searching(Node *head, int value)
{
    Node *temp = head;
    int position = 0;
    while (temp != nullptr)
    {
        if (temp->data == value)
        {
            cout << value << " found at position " << position << ".\n";
            return;
        }
        temp = temp->next;
        position = position + 1;
    }
    cout << value << " not found in the list.\n";
}

```

```

void last_node(Node *head)
{
    if (head == nullptr)
    {
        cout << "List is empty.\n";
        return;
    }
    Node *temp = head;
    while (temp->next != nullptr)
    {
        temp = temp->next;
    }
}

```

```
}  
  
cout << "Last node value: " << temp->data << "\n";  
}
```

```
void previous_of_last_node(Node *head)  
{  
    if (head == nullptr || head->next == nullptr)  
    {  
        cout << "List has fewer than 2 nodes. No previous-of-last node.\n";  
        return;  
    }  
    Node *temp = head;  
    while (temp->next->next != nullptr)  
    {  
        temp = temp->next;  
    }  
    cout << "Previous of last node value: " << temp->data << "\n";  
}
```

```
void list_size(Node *head)  
{  
    int count = 0;  
    Node *temp = head;  
    while (temp != nullptr)  
    {  
        count = count + 1;  
        temp = temp->next;  
    }  
}
```

```

    cout << "List size: " << count << "\n";
}

void reversePrintHelper(Node *node, Node *head)
{
    if (node == nullptr)
    {
        return;
    }
    reversePrintHelper(node->next, head);
    cout << node->data;
    if (node != head)
    {
        cout << " -> ";
    }
}

```

```

void reversePrint(Node *head)
{
    if (head == nullptr)
    {
        cout << "List is empty.\n";
        return;
    }
    cout << "Reversed List: ";
    reversePrintHelper(head, head);
    cout << "\n";
}

```

/*

----- Insert Operations -----

10 inserted at last.

20 inserted at last.

30 inserted at last.

5 inserted at first.

List: 5 -> 10 -> 20 -> 30

15 inserted at position 2.

List: 5 -> 10 -> 15 -> 20 -> 30

25 inserted after 20.

List: 5 -> 10 -> 15 -> 20 -> 25 -> 30

----- Search / Info Operations -----

25 found at position 4.

100 not found in the list.

Last node value: 30

Previous of last node value: 25

List size: 6

Reversed List: 30 -> 25 -> 20 -> 15 -> 10 -> 5

----- Delete Operations -----

5 deleted from first.

List: 10 -> 15 -> 20 -> 25 -> 30

30 deleted from last.

List: 10 -> 15 -> 20 -> 25

15 deleted from position 1.

List: 10 -> 20 -> 25

25 deleted from the list.

List: 10 -> 20

List size: 2

----- Final State -----

List: 10 -> 20

Reversed List: 20 -> 10*/